

流式套接字聊天程序通信协议说明

(Simple Chatroom Communication Protocol — SCCP v1.0)

姓名：谢小珂 学号：2310422 专业：物联网工程

1. 协议概述

1.1 基本信息

- 协议名称：SCCP v1.0 (Simple Chatroom Communication Protocol)
- 传输层：TCP (SOCK_STREAM, 面向连接、字节流)
- 默认端口：9090
- 字符编码：UTF-8 或当前系统控制台编码 (实现已统一中文提示)
- 支持语言：中文、英文及多语言文本

1.2 设计原则

- 简单易实现：以“单行文本 + `\n`”作为消息边界，直接可用、便于调试；
- 多用户并发：服务器一连一线程，客户端收发分离；
- 基本可靠性：依托 TCP 的有序可靠；应用层做空行/长度校验；
- 可读性：所有显示消息带**用户标识**与**秒级时间戳**。

2. 连接管理协议

2.1 建立连接

- 客户端与服务器完成 TCP 三次握手；
- 服务器 `accept()` 成功后，为该连接**创建独立会话线程**处理收发。

2.2 身份注册 (HELLO)

- 首包：客户端必须先发送**用户名**（一行文本，以 `\n` 结尾）；
- 校验：空/仅空白则置为 `"guest"`；建议最长 ≤ 127 字节；
- 存档：服务器将该用户名与此连接绑定，用于后续群发展示。

2.3 连接确认 (SYSTEM)

- 服务器在收到合法用户名后发送两类**中文系统消息**（均带时间戳）：

1. 仅回给本人：

1 | [系统] <用户名> 已成功建立连接。[YYYY-MM-DD HH:MM:SS]\n

2. 广播给其他在线用户：

1 | [系统] <用户名> 加入了聊天室。[YYYY-MM-DD HH:MM:SS]\n

3. 消息传输协议

3.1 数据包格式示意图

SCCP v1.0 采用“单行文本 + 换行”作为消息边界；客户端每条消息都以 \n 结束；服务器在广播前组装展示文本并在末尾补上 \n。

字段	类型	长度/取值	说明
消息内容	字符串	≤ 1023 字节	客户端原始输入（getline 已去除行尾）
结束符	1 字节	\n (0x0A)	客户端主动追加的换行分隔符

• 客户端发送 (C→S)：

1 | <消息内容> + "\n"

• 服务器广播 (S→C)：

1 | [<用户名>] <消息内容> [YYYY-MM-DD HH:MM:SS] + "\n"

或

1 | [系统] <提示内容> [YYYY-MM-DD HH:MM:SS] + "\n"

3.2 消息类型与显示

类型	方向	传输内容（原始）	屏幕展示（格式化）	说明
HELLO	C→S	<用户名>\n	——	首包注册昵称
USER	C→S	<聊天>\n	服务器转发： [<用户名>] <聊天> [时间]\n	群聊消息（服务器加时）
SYSTEM	S→C	文本（服务器生成）	[系统] <提示> [时间]\n	加入/离开/确认/提示
LOCAL	本地	——	[我] <聊天> [时间]\n	仅客户端本地回显

注：客户端输入 "quit" 不作为 USER 发送；用作本地退出指令。

3.3 边界处理细节

- 分隔符：以 单个 `\n` 作为消息边界；
- 客户端发送：主线程读取一整行（`std::getline` 已去掉行尾换行），随后主动追加 `\n` 再 `send()`；
- 服务器接收：每次读取到一行后，先移除末尾的 `\r/\n`（兼容 `\r\n`），再进入格式化；
- 服务器广播：展示文本（`[用户名]` `内容` `[时间]` / `[系统]` `提示` `[时间]`）末尾统一追加 `\n` 后发送；
- 客户端显示：接收线程原样打印服务器广播；本地回显由客户端在发送后自行打印 `[我]` `内容` `[时间]`。

3.4 长度与编码

- 最大长度：建议消息体 ≤ 1023 字节（含结尾 `\n` 不超过 1024）；
- 策略：超长消息采用“截断”策略（或丢弃，二选一需全程一致；与你当前实现一致建议“截断”）；
- 空消息：丢弃，不转发；
- 编码：按系统控制台编码/UTF-8 显示；无法识别字符按原字节输出。

3.5 时间戳规范

- 格式：`YYYY-MM-DD HH:MM:SS`；精度：秒；时区：本地系统时区；
- 服务器：对 **USER/SYSTEM** 广播在末尾追加 `[时间]`；
- 客户端：对 **LOCAL** 本地回显在末尾追加 `[时间]`；
- 效果：所有可见消息末尾时间戳样式一致，便于核对顺序。

4. 数据包处理规则

4.1 客户端处理流程

发送路径（主线程）：

1. `std::getline` 读取 `line`；
2. 若 `line == "quit"`：
 - 主线程不再发送消息，直接 `closesocket(sock)` 关闭连接；
 - 置 `g_running = false` 结束主循环；
 - 接收线程在下次 `recv()` 返回 `<= 0`，打印：`[系统]` 与服务器的连接已断开。 ，随后结束；
 - 主线程 `join()` 等接收线程退出，清理资源并退出程序。
3. 否则（普通消息）：
 - `send(sock, (line + "\n").c_str(), ...)`；
 - 本地 UX：调用 WinAPI（`GetConsoleScreenBufferInfo` / `SetConsoleCursorPosition` / `FillConsoleOutputCharacterA`）擦除上一行原始输入，仅打印

```
1 | [我] <line> [YYYY-MM-DD HH:MM:SS]\n
```

- 继续下一轮输入。

接收路径（接收线程）：

- 阻塞 `recv()`，将服务器发来的 **SYSTEM/USER** 文本**原样打印**（其中已包含用户名/提示与时间戳）；
- 当 `recv <= 0`（服务器关闭或网络异常）时，打印

```
1 | [系统] 与服务器的连接已断开。
```

并结束线程。

4.2 服务器处理流程

1. **首包用户名**：第一条收到的行作为**用户名**；空则置 `"guest"`；
2. **连接确认与加入广播**：
 - `send()` 给**该客户端**：
[系统] <用户名> 已成功建立连接。[时间]\n
 - `broadcast()` 给**其他客户端**：
[系统] <用户名> 加入了聊天室。[时间]\n

3. 主循环接收聊天：

- `recv()` 到一行 → 去尾 `\r/\n` → 若非空：

```
1 | "[" + uname + "]" " " + text + " [" + now_str() + "]\n"
```

`broadcast(...)` 到**其他客户端**（排除发送者）；

4. **断连与离开广播**：当 `recv <= 0` 或线程结束：
 - 从在线表移除该连接（`vector<Client>` + `mutex`）；
 - `broadcast()`：
[系统] <用户名> 离开了聊天室。[时间]\n
 - `closesocket(cli)`，线程退出。

5. 错误处理与异常协议

- **连接异常**：任一端 `recv <= 0` 视为断开；服务器清表并广播“离开”；客户端提示“连接已断开”；
- **首包违规**：若未先发昵称即发送其他内容，服务器可直接关闭（或按收到内容作为昵称；二者择一，需与实现一致）；
- **空/超长消息**：空消息丢弃；超长**截断或丢弃**（选一并在报告中声明，建议“截断”）；
- **端口占用**：服务器使用 `SO_REUSEADDR` 并允许改端口（默认 9090）；
- **限流（可选）**：可按连接设置速率阈值，超限忽略或断开（代码中预留扩展点）。

6. 并发与线程安全协议

6.1 客户端线程模型

- **主线程**：读取输入、发送消息、处理 `quit`、本地“[我]”回显（擦行+打印）；
- **接收线程**：阻塞 `recv()` 并打印服务器 **SYSTEM/USER**；监控断连并提示。

6.2 服务器线程模型

- **主线程**：`bind/listen/accept`；每个新连接 `std::thread(client_thread).detach()`；
- **会话线程**：首包昵称 → 确认/加入广播 → 循环接收/广播 → 断连清理/离开广播；
- **共享数据**：`std::vector<Client>` 存在线用户；`std::mutex` 保证并发安全；`broadcast()` 负责群发。

6.3 同步与优化建议

- **同步**：`broadcast()` 与在线表操作使用 `std::lock_guard<std::mutex>`；
- **优化**：为减少持锁时间，可在广播前复制 **socket 列表**后解锁再逐个发送（不改变协议层行为）。

版本：SCCP v1.0

适用：简易多人在线聊天室

技术基础：Winsock2 基础接口（`socket/bind/listen/accept/connect/send/recv/closesocket`）